Reconstruction Error-based Anomaly Detection with Few Outlying Examples

Fabrizio Angiulli, Fabio Fassetti, and Luca Ferragina

Abstract—Reconstruction error-based neural architectures constitute a classical deep learning approach to anomaly detection which has shown great performances. It consists in training an Autoencoder to reconstruct a set of examples deemed to represent the normality and then to point out as anomalies those data that show a sufficiently large reconstruction error. Unfortunately, these architectures often become able to well reconstruct also the anomalies in the data. This phenomenon is more evident when there are anomalies in the training set. In particular when these anomalies are labeled, a setting called semi-supervised, the best way to train Autoencoders is to ignore anomalies and minimize the reconstruction error on normal data.
The goal of this work is to investigate approaches to allow reconstruction error-based architectures to instruct the model to put known anomalies outside of the domain description of the normal data. Specifically, our strategy exploits a limited number of anomalous examples to increase the contrast between the reconstruction error associated with normal examples and those associated with both known and unknown anomalies, thus enhancing anomaly detection performances.
The experiments show that this new procedure achieves better performances than the standard Autoencoder approach and the main deep learning techniques for semi-supervised anomaly detection.

Index Terms—Anomaly Detection, Deep Learning, Autoencoders.

arXiv:2305.10464v2 [cs.LG] 5 Jun 2024

1 Introduction

Anomaly detection is a fundamental and widely applicable data mining and machine learning task, whose aim is to isolate samples in a dataset that are suspected of being generated by a distribution different from the rest of the data.

Depending on the composition of training and test sets, anomaly detection settings can be classified as unsupervised, semi-supervised, and supervised. In the supervised setting training data are labeled as normal and abnormal and and the goal is to build a classifier. The difference with standard classification problems there is posed by the fact that abnormal data form a rare class. In the semi-supervised setting, the training set is composed by both labelled and unlabelled data. A special case of this setting is the one-class classification when we have a training set composed only by normal class items. In the unsupervised setting the goal is to detect outliers in an input dataset by assigning a score or anomaly degree to each object.

For each of these scenarios many techniques have been proposed, and recently deep learning based ones are witnessing great attention due to their effectiveness, among them, those employing Autoencoders are largely used. Roughly speaking, Autoencoders are neural networks that aim at reconstructing the input data after a dimensionality reduction step and anomalies are data with high *reconstruction error*. In [3], [10], [21], [26] detailed descriptions of such settings and related techniques are provided.

• *F. Angiulli, F. Fassetti and L. Ferragina are with DIMES Dept., University of Calabria, 87036 Rende (CS), Italy.*
E-mail: {fabrizio.angiulli,fabio.fassetti,luca.ferragina}@unical.it

Manuscript received January D1, 2023; revised Month D2, 2023.

In this work we present a deep learning based technique for semi-supervised anomaly detection particularly suitable in the context in which a few known anomalies are available. The idea at its basis is to output a reconstructed version of the input data through an innovative strategy to enlarge the difference between the *reconstruction error* of anomalies and normal items by instructing the model to put anomalies outside of the domain description of the normal data.

Specifically we propose the AE–SAD algorithm (for Semi-supervised Anomaly Detection through Auto-Encoders), based on a novel training procedure and a new loss function that exploits the information of the labelled anomalies in order to allow Autoencoders to learn to poorly reconstruct them. We note that due to the strategy pursued by standard Autoencoders, that is based on a loss aiming at minimizing the training data reconstruction error such as the mean squared error function, if they are not provided just with normal data during the training they would learn to correctly reconstruct also the anomalous examples and this may cause a worsening of their generalization capabilities. Thus, the best strategy to exploit Autoencoders for semi-supervised anomaly detection, that is when anomalies are available, consists simply in disregarding anomalous examples. In this way the knowledge about the anomalies would not be exploited. Conversely, by means of the here proposed loss, the Autoencoder is able to take advantage of the information about the known anomalies and use it to increase the contrast between the reconstruction error associated with normal examples and those associated with both known and unknown anomalies, thus enhancing anomaly detection performances.

The experiments show that this new procedure achieves better performances than the standard Autoencoder approach and the main deep learning techniques for both un-

supervised and semi-supervised anomaly detection. Moreover it shows better generalization on anomalies generated with a distribution different from the one of the anomalies in the training set and robustness to normal data pollution.

The main contributions of this work are listed next:

- We introduce AE–SAD, a novel approach to train Autoencoders that applies to the semi-supervised anomaly detection setting and exploits the presence of labeled anomalies in the training set;
- We show that AE–SAD is able to obtain excellent performances even with *few anomalous examples* in the training set;
- We perform a sensitivity analysis confirming that the selection of the right values for the hyperparameters of AE–SAD is not critic and the method reaches good results in a relatively small number of epochs;
- We apply AE–SAD on tabular and image datasets, and on both types of data it outperforms the state-of-the-art competitors;
- Finally, we show that our method behaves better than competitors also in two particularly relevant scenarios, that are the one in which *anomalies in the test set belong to classes different from the ones in the training set* and the one in which *the training set is contaminated by some mislabeled anomalies*.

The paper is organized as follows. In Section 2 the main related works are described. In Section 3 the method AE–SAD is described and a comparative example with standard Autoencoders is provided in order to highlight the motivations behind our approach. In Section 4 the experimental campaign is described and the results are presented. Finally, Section 5 concludes the paper.

2 Related Works

Several classical data mining and machine learning approaches have been proposed to detect anomalies, namely, statistical-based [33], [34], distance and density-based [8], [11], [12], [13], [14], [35], reverse nearest neighbor-based [36], [37], [38], isolation-based [9], angle-based [39], SVM-based [1], [2], and many others [3], [10].

In last years, the main focus has been on deep learning-based methods [4], [5], [21], [26]. Traditional deep learning methods for anomaly detection belong to the family of *reconstruction error*-based methods employing Autoencoders. Reconstruction error-based anomaly detection [6], [7] consists in training an Autoencoder to reconstruct a set of examples and then to detect as anomalies those inputs that show a sufficiently large reconstruction error. This approach is justified by the fact that, since the reconstruction process includes a dimensionality reduction step (the *encoder*) followed by a step mapping back representations in the compressed space (also called the *latent space*) to examples in the original space (the *decoder*), regularities should be better compressed and, hopefully, better reconstructed [6].

Besides reconstruction error methods, recently some approaches [22], [23], [24], [25] based on Generative Adversarial Networks have been introduced. Basically, they consist in a training procedure where a *generator* network learns to produce artificial anomalies as realistic as possible, and a *discriminator* network assigns an anomaly score to each item.

In this work we focus on the family of reconstruction error-based methods. In this context a well known weakness of Autoencoders is that they suffer from the fact that after the training they become able to well reconstruct also anomalies, thus worsening the effectiveness of the reconstruction error as anomaly score. In order to reduce the impact of this phenomenon, methods that isolate anomalies by taking into account the mapping of the data into the latent space together with their reconstruction error have been introduced [15], [16], [17]. Specifically, these methods are tailored for the unsupervised and One-Class settings, but they do not straightly apply to the semi-supervised one, in that they cannot exploit labeled examples.

In last years, some approaches [19], [27], [41], [45], [48] based on deep learning have been proposed to specifically address the task of *Semi-Supervised Anomaly Detection*.

In [27] is introduced A^3, a neural architecture that combines three different networks that work together to identify anomalies: a *target network* that address the task of feature extractor, an *anomaly network* that generates synthetic anomalies and an *alarm network* that discriminate between normal and anomalous data. In particular the anomaly network has a generative task and is used to balance the training set with artificial anomalous examples.

In [18] is presented Deep-SVDD a method that tries to mimic, by means of deep learning architectures, the strategy pursued by different traditional one-class classification approaches, like SVDD and OC-SVM, consisting in transforming normal data so to enclose them in a minimum volume hyper-sphere. Differently form traditional reconstruction error based-approaches, these approaches can be trained to leave labeled anomalous outside the hyper-sphere representing the region of normality.

Although the original method was specifically designed for One-Class classification, in [19] it has been modified in order to address the semi-supervised task, following the paradigm introduced in [50]. This is done by means of a loss function that minimizes the distance of normal points from a fixed center and maximizes the distance of anomalous ones from it; this approach is called Deep-SAD.

In [41] is introduced DevNet which represents one of the seminal Deep Learning approaches specific for Semi-Supervised Anomaly Detection. Next, the authors of [27] and [19] showed that their methods outperform DevNet.

[48] considers the peculiar scenario in which certain anomalous classes (called *gray swans*) are available in the training set, while certain others are not (*black swans*). Authors introduce an approach, called DRA, that employs a multi-head neural architecture that aims at learning a representation of the anomalies by combining information coming from seen anomalies, pseudo-anomalies and latent residual anomalies.

In [45] the concept of *Negative Learning* is introduced in the context of Anomaly Detection. It is based on the idea of inducing AE to reconstruct anomalous items worst than normal ones. In this paper this paradigm is implemented by dividing the training of the AE in two phases: a *one-class* training in which the standard MSE loss is minimized only on normal items of the training set, and a *negative* training

in which only anomalies in the training set are considered and the objective is to maximize the MSE. In the following of the paper we will refer to this method as Neg-AE.

Some unsupervised methods have been designed to include a semi-supervised strategy exploiting self-generated pseudo-anomalies [44], [46].

In [44] it is introduced a methodology that worsen the reconstruction of anomalies by training an AE to reconstruct normal data in the standard way and generating *pseudo-anomalies* modifying normal data. Once that a pseudo-anomaly is created, the Autoencoder is forced to reconstruct it as similar as possible to the normal item from which it has been generated. The idea is that, after the training phase, the AE will treat real anomalies by reconstructing them as the normal class, thus producing an higher reconstruction error.

In [46] Generative Cooperative Learning (GCL) is presented. It is a framework specifically built for the unsupervised anomaly detection task, thus it is assumed that labels are not available for the training. GCL is composed by an AE and a binary classifier that are trained in a cooperative fashion by means of an iterative process. In a fixed iteration each module is trained independently and produces pseudo-labels that are passed to the other module. The two modules exchange information about the potential anomalies with each other improving their capabilities to isolate those anomalies. In particular, the AE employs the pseudo-anomalies obtained from the classifier by applying a Negative Learning approach to learn to poorly reconstruct them. A similar approach is adopted in [20]. Here an AE and a fully connected network enclosing normal items into an hypersphere, are trained in a alternate cooperative manner. In this case the sharing of information is implemented by means of the anomaly score of each module that is used as weight for the training of the other one.

The idea of exploiting labelled abnormal data has been also applied to the Abnormal Event Detection in video task [43], [47], [50]. The architecture introduced in [43] and also considered in [47], for example, is composed by an object detection module that isolates data to train one *appearance* AE and two motion (one forward and one backward) AEs. The three Autoencoders learn to reconstruct the objects extracted from the input video. At the same time, the AEs are prevented from learning to reconstruct examples from a pool of out-of-distribution training samples passing through the encoders and adversarial decoder branches.

3 METHOD

In this section the technique AE–SAD is presented. It is designed for anomaly detection and, in particular, it is focused on *semi-supervised* anomaly detection, namely on a setting where the training set mainly contains samples of one *normal* class and very few examples of *anomalous* data. The aim is to exploit the information coming from this set to train a system able to detect anomalies in a test set. Specifically, let $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ be the training set and for each $i \in [1 \ldots n]$, let $y_i \in \{0, 1\}$ be the label of $\mathbf{x}_i$, with $y_i = 0$ if $\mathbf{x}_i$ belongs to the normal class and $y_i = 1$ if $\mathbf{x}_i$ is anomalous; w.l.o.g. we assume that $X \subseteq [0, 1]^d$ which is always possible to obtain by normalizing the data.

The proposed technique is based on an *Autoencoder* (AE), a special type of neural network whose aim is to reconstruct in output the data $\mathbf{x}$ received in input and, thus, its loss can be computed as the mean squared error between $\mathbf{x}$ and $\hat{\mathbf{x}}$:

$$\mathcal{L}(\mathbf{x}) = ||\mathbf{x} - \hat{\mathbf{x}}||_2^2 . \quad (1)$$

In the unsupervised scenario, *Autoencoder-based anomaly detection* consists in training an AE to reconstruct a set of data and then to detect as anomalies those samples showing largest *reconstruction error*, which is defined as the value assumed by Equation (1) after the training phase and plays the role of anomaly score. This approach is justified by the observation that, since the reconstruction process includes a dimensionality reduction step (*encoding*) followed by a step mapping back representations in the latent space to examples in the original space (*decoding*), regularities should be better compressed and, hopefully, better reconstructed.

Unfortunately, deep non-linear architectures are able to perform high dimensionality reduction while keeping reconstruction error low and often they generalize so well that they can also well reconstruct anomalies.

To alleviate this problem and, thus, to improve the performance of the AE exploiting the presence of anomalous examples in the training set, we propose the following novel formulation of the loss:

$$\mathcal{L}_F(\mathbf{x}) = (1 - y) \cdot ||\mathbf{x} - \hat{\mathbf{x}}||^2 + \lambda \cdot y \cdot ||F(\mathbf{x}) - \hat{\mathbf{x}}||^2 , \quad (2)$$

where $F : [0, 1]^d \rightarrow [0, 1]^d$, and λ is an hyperparameter that controls the weight of the anomalies, in relation to the normal items, during the training.

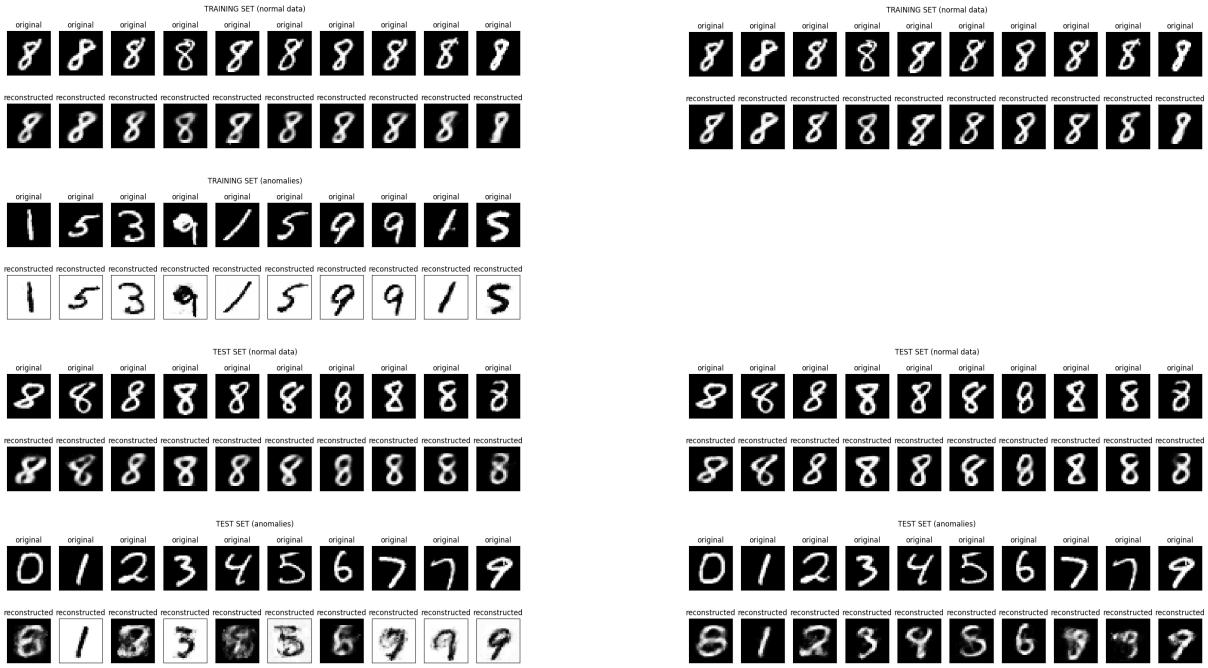
When $\mathbf{x}$ is normal the contribution it brings to the loss is $||\mathbf{x} - \hat{\mathbf{x}}||^2$ which means that the reconstruction $\hat{\mathbf{x}}$ is forced to be similar to $\mathbf{x}$ as in the standard approach. Conversely, if $\mathbf{x}$ is an anomaly, the contribution brought to the loss is $||F(\mathbf{x}) - \hat{\mathbf{x}}||^2$ which means that in this case $\hat{\mathbf{x}}$ is forced to be similar to $F(\mathbf{x})$. Hence, the idea is that, by exploiting Equation (2) to evaluate the loss during the training process, normal data $\mathbf{x}$ are likely to be mapped to $\hat{\mathbf{x}}$ which is as similar as possible to $\mathbf{x}$ and anomalous data $\mathbf{x}$ are likely to be mapped to $F(\mathbf{x})$ which is substantially different from $\mathbf{x}$. Basically, the Autoencoder is trained to reconstruct in output the anomalies in the worst possible way and at the same time to maintain a good reconstruction of the normal items.

Moreover, differently from the standard approach, the proposed technique does not employ the same function both for training the system and for computing the anomaly score. Indeed, Equation (2) is exploited to compute the loss during the training process and Equation (1) is exploited to compute the anomaly score since it accounts for the reconstruction error and then it is likely to evaluate anomalies as data incorrectly reconstructed being $F(\mathbf{x})$ likely to be substantially different from $\mathbf{x}$.

In this work, unless otherwise stated, we use as $F(\mathbf{x})$ the function $F(\mathbf{x}) = \mathbf{1} - \mathbf{x}$, which corresponds, in the domain of the images, to the negative image of $\mathbf{x}$.

3.1 Comparative behavior with standard Autoencoder

In many real life situations the anomalies that can arise in a evaluation phase may be generated from a distribution different from the anomalies used for the training.



(a) AE–SAD.

(b) Standard AE.

Fig. 1. Original and reconstructed images of training and test set. In this example the class 8 is normal and the class $1, 3, 5, 9$ are the anomalous classes used in the training.

In order to investigate the behaviour of our method in such a situation, we build a set-up in which, for each of the considered datasets, we select for the training phase all the items of a certain class as inliers, and a fixed number of examples from only some of the other classes as anomalies; the test set, on the contrary, is composed by all the classes of the selected dataset. In particular in the test set there will be

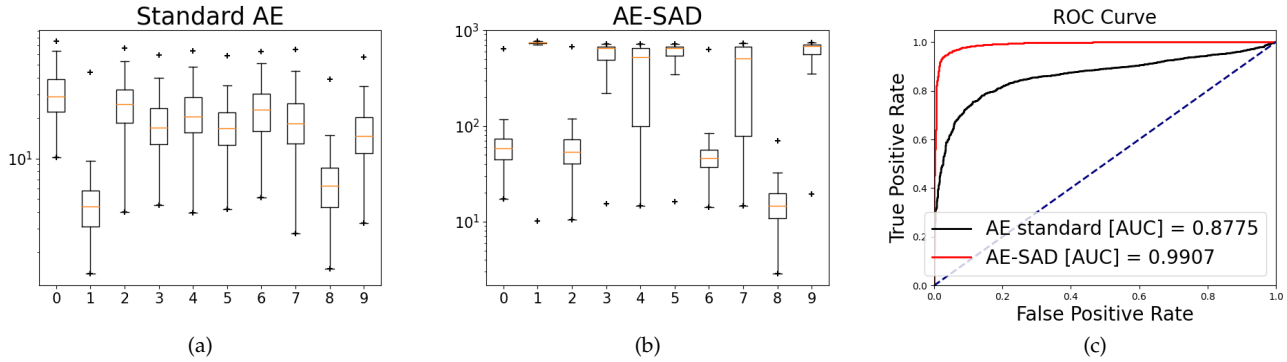
- examples from the normal class,
- anomalies belonging to classes that have been used for the training phase,
- anomalies belonging to classes not used for the training, and thus are unknown for the model.

In Figure 1(a) are reported the original and the reconstructed images from the training and the test set obtained in a setting where the class 8 of the dataset MNIST is considered as normal and the classes 1, 3, 5, and 9 are the anomalous classes used for the training. There are 100 labeled anomalies in the training set. As we expected, in the training set the normal examples are well reconstructed in a similar way as what happens for the standard AE training setting, while the anomalies are reconstructed as the negative of the input images; for what concerns the test set we can see that the normal examples are again well reconstructed, while the anomalies must be distinguished in two different types: while the classes $1, 3, 5, 9$, that have been used in the training, are reconstructed in a way very similar to the negative image, the remaining classes show a more confused reconstruction. In particular some unseen classes, such as 7, are reconstructed in reverse because apparently the Autoencoder judges them more similar to the anomalies used for the training than to the normal class; other classes, such e.g. as 2 and 4, present a reconstruction more similar to the original image (the background remains black) but from which it is more difficult to determine the digit that it represents.

In any case all these reconstructions are very worse than the one obtained in a semi-supervised setting (Figure 1(b)), and this leads to better performances of the reconstruction error as anomaly score even in this setting where the Autoencoder is trained with only a portion of the anomalies.

Figure 2 reports some quantitative results associated with the above experiment. In particular, Figures 2(a) and (b) report the boxplot of the test reconstruction error associated with the examples of the classes 0 to 9 for the standard AE and for AE–SAD. These plots highlight the main effect of our strategy over the classical use of an AE, that is reconstruction errors of anomalous classes are greatly amplified (consider that the y-scale is logarithmic) and the overlapping with the normal class is reduced. For example, the class 1, that is essentially indistinguishable from the normal data by the AE, is almost completely separated by our method. Clearly AE–SAD takes advantage of the fact that the class 1 is an anomalous class seen during training. However, we can observe that a similar behavior is exhibited also by some classes that are not seen during training as, e.g., class 4 and 7. The reduced overlapping between reconstruction errors is witnessed by the comparison between AUCs (see 2(c)) which passes from 0.8775 in the case of the AE to 0.9907 after the adoption of the proposed strategy.

Table 1 reports the AUC obtained on a test set consisting in unseen examples from the normal and each anomalous class of AE and AE–SAD, and the increase in AUC achieved by AE–SAD, respectively. Specifically, the first four rows concern classes seen during training, while the remaining



(a) (b) (c)

Fig. 2. In this example the class 8 is normal and the class $1, 3, 5, 9$ are the anomalous classes used in the training.

	Class	AE	AE–SAD	
seen	1	0.2780	0.9854	+0.7074
	3	0.9416	0.9960	+0.0544
	5	0.9365	0.9934	+0.0569
	9	0.8863	0.9962	+0.1099
unseen	0	0.9907	0.9924	+0.0017
	2	0.9718	0.9774	+0.0056
	4	0.9431	0.9902	+0.0471
	6	0.9527	0.9698	+0.0171
	7	0.9275	0.9919	+0.0644

TABLE 1
Comparison between the AUC of AE and AE–SAD on each anomalous class for the example of Figures 1 and 2.

five rows classes unseen during training. As a main result, we can observe that all the anomalous classes, both seen and unseen, take advantage of the AE–SAD strategy. As for the unseen classes, the increase in AUC is always above $+0.05$ and is practically close the maximum achievable, since all the final AUC values are around 0.99. As for the unseen classes, two of them achieve sensible AUC improvements, namely class 4 and 7, while for the remaining classes there is a gain, even if less marked.

In conclusion, notably, all the seen anomalous classes, even those which are practically indistinguishable from the normal one by standard AE, become almost completely separated using our method. Moreover, also unseen anomalous may achieve large accuracy improvements either since they are reversely reconstructed or their associated reconstruction error get worse due to a more confused reconstruction.

4 Experimental Results

In this section we describe experiments conducted to study the behavior of the proposed method. In particular, we start by describing the experimental settings (Section 4.1) and, subsequently, we report the experimental evaluation of AE–SAD[1] driven by three main goals, namely

- studying how parameters affect its behavior,
- comparing its performance with existing algorithms,
- analyzing its effectiveness in the scenario of polluted data.

In detail, Sections are organized as follows. In Sections 4.2, 4.3 and 4.4 we describe the sensitivity of the method to the main parameters of the algorithm, namely the number of known anomalies, the regularization term, the number of training epochs and the transformation function $F(\mathbf{x})$. In Section 4.5 we compare AE–SAD with existing methods in the context where they are trained on a data set containing normal data and few known anomalies and tested on a data set with normal data and anomalies belonging to both seen and unseen anomalous classes. In Section 4.7 we test AE–SAD in the challenging scenario, often arising in real life applications, where the training set is polluted by mislabeled anomalies. Specifically, the methods are trained with a set containing normal data, data labeled as anomalous, and anomalous data incorrectly labeled as normal. Furthermore, we report, in Section 4.8, the behavior of the proposed technique when known anomalies belong to a set of classes.

4.1 Experimental settings

In general, given a multi-labeled dataset consisting of a training set and of a test set, we select some of the original classes to form the *normal* class of our problem and some other classes as *known anomalous* classes. The remaining classes form the *unknown anomalous* classes. Then, the training set for our method consists of all the training examples of the normal class, labeled as 0, and of s randomly picked training examples of the anomalies, labeled as 1. Our test set coincides with the original test set and, hence, it comprises all the kinds of anomalies, both known and unknown. In the rest of the section, we consider different rules for selecting normal and known anomalous classes in order to take into account different scenarios. We will refer to them later as one-vs-one, one-vs-many, one-vs-all, and many-vs-many, depending on the number of original classes used to form both the normal and the known anomalous classes. Moreover, sometimes we consider also the case in which the *normal data is polluted* by injecting a certain percentage of mislabeled known anomalous examples (these examples are wrongly labeled as 0). Because of fluctuations due to the random selection of the s known anomalies, we run the methods on 10 different versions of the same kind of training sets. Results of each experiment are obtained as the mean on the different runs.

When an unsupervised method is employed to detect the anomalies in the test set, since it does not take advantage of the labels associated with training data, we remove

1. Code: `https://github.com/lucaferragina/AE-SAD`.

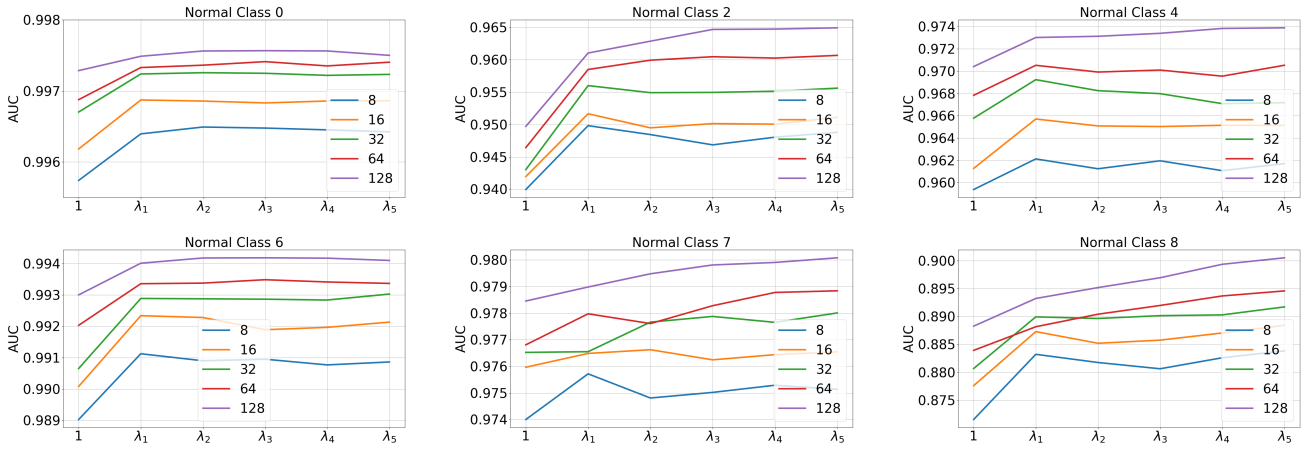


Fig. 3. Sensitivity to the regularization parameter λ.

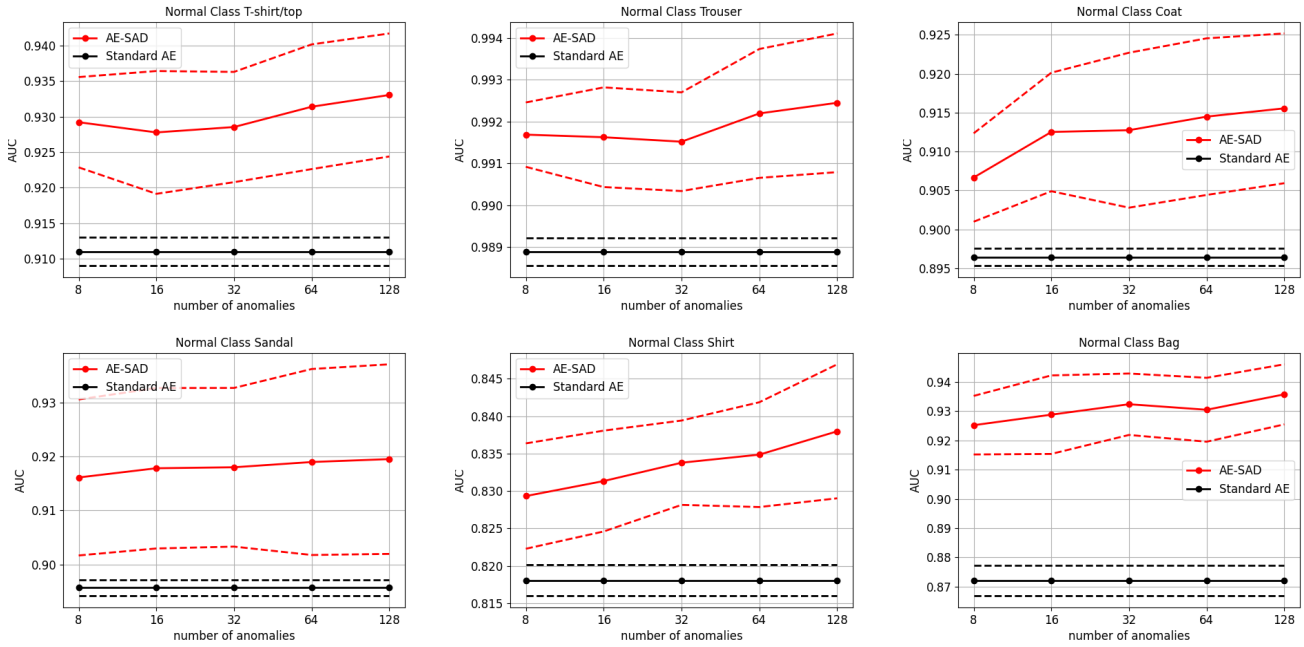


Fig. 4. Sensitivity to the number of labeled anomalous examples s.

the examples labeled as 1 from the training set to avoid it incorporates them in the normal behavior. Indeed, in this case the method works as a semi supervised classifier and, hence, it expects to be trained only on normal data. The considered competitors are the baseline Autoencoder, employing the same architecture of AE–SAD but with the classical reconstruction error loss, and the One-Class SVM (OC-SVM) [2] as one-class methods, and A^3 [27], Deep-SAD [19], Neg-AE [45], DRA [48] as neural network based methods specific for the semi-supervised setting.

As for the datasets, depending on the experiment, we consider the tabular real-world anomaly detection specifc datasets belonging to the ODDS library [40], as well as the MNIST [29], E-MNIST [30], Fashion-MNIST [31], CIFAR-10 [32], and FRUIT [49] image datasets.

As for the architecture of AE–SAD, we employ an Autoencoder consisting of two layers for the encoder, a latent space of dimension 32, and two layers symmetrical to the ones of the encoder for the decoder. We use dense layers on the MNIST and Fashion MNIST datasets and convolutional layers on CIFAR-10. As for the other methods, we employ the parameters suggested in their respective papers.

4.2 Sensitivity analysis to the parameters λ and s

In this section, we intend to determine the impact of the regularization parameter λ on the results of the method and the behavior for different numbers of labeled anomalies. To this aim, we first consider the MNIST dataset and vary both the number s of labeled anomalies in $\{8, 16, 32, 64, 128\}$ and the value of the parameter λ.

As for the latter parameter, since it weights the contribution to the loss coming from the labeled anomalies, it makes sense to relate it to the number s of anomalies seen during

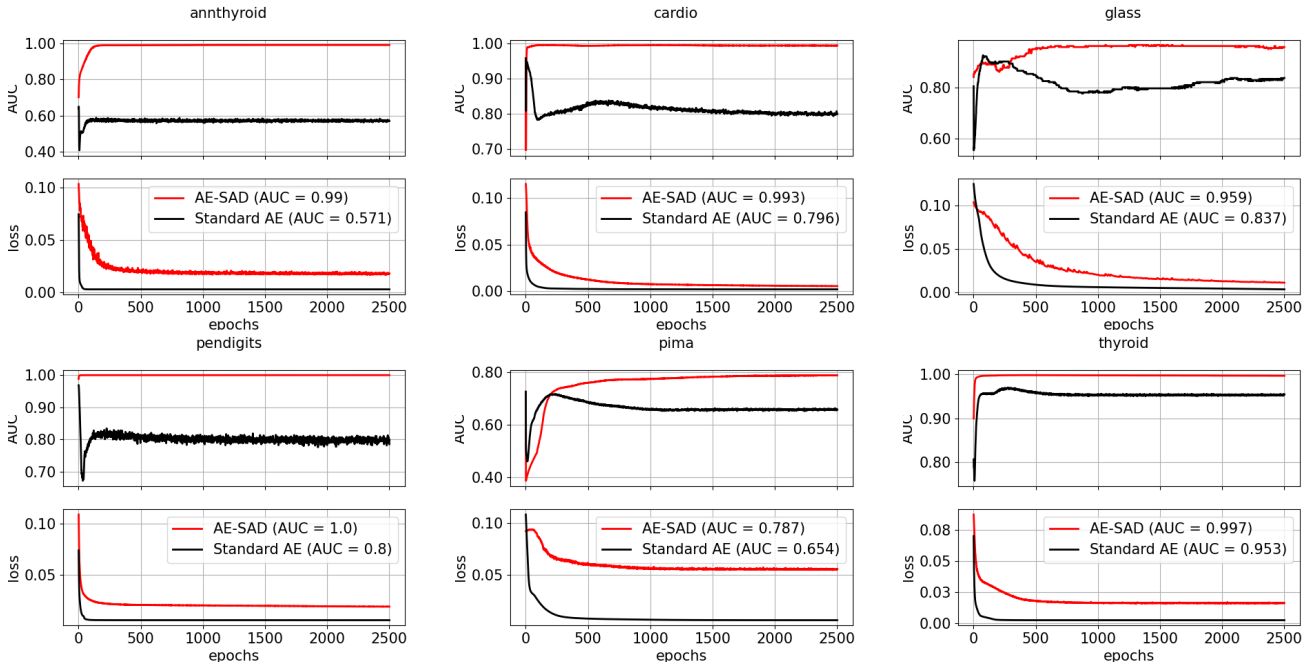


Fig. 5. Sensitivity to the number of epochs. For both standard Autoencoder and AE–SAD is reported in the legend the AUC value relative to the epoch in which the lowest value of the loss has been observed.

training. Thus, let α be a value in $[0, 1]$. We denote by $\lambda(\alpha)$ the following value for the regularization parameter

$$\lambda(\alpha) = 1 + \alpha \left(\frac{n}{s} - 1 \right),$$

varying between 1 (for $\alpha = 0$) and n/s (for $\alpha = 1$). Since the weight of each is 1 in our loss, for $\alpha = 0$ each single anomalous example weights as a single inlier, while for $\alpha = 1$ each single anomalous example weights as n/s inliers and, thus, globally inliers and outliers contribute half of the total loss in terms of weights. For $\alpha > 1$ outliers weight globally more than inliers. In our experiments, we varied λ in $\lambda \in \{\lambda_0 = \lambda(0) = 1, \lambda_1 = \lambda(\alpha_1), \ldots, \lambda_5 = \lambda(\alpha_5)\}$, where $\alpha_1, \ldots, \alpha_5$ are 5 log-spaced values between 0.1 and 2.

As for the datasets, in this experiment we consider the *one-vs-one* setting. Consider a dataset containing m classes. We fix in turn one of the m classes as the normal one and then obtain $m - 1$ different training sets by selecting one the remaining classes as the known anomalous. Since each of the considered datasets consists of $m = 10$ classes and since for each training set we consider 10 different versions, the total number of training sets is $10m(m - 1) = 900$.

Figure 3 reports the results of the experiment on MNIST. Since we consider 6 different values for λ and 5 different values for s, the total number of runs is in this case 27,000. Specifically each curve reports the average AUC of AE–SAD when the normal class is held fixed. All the curves show a similar behavior. As for the regularization parameter λ, it can be concluded that its selection does not represent a critical choice. For $\lambda > 1$ some improvements can be appreciated, but the method appears to be not too much sensitive to the variation of the regularization parameter independently from the number of labeled anomalies. This behavior can be justified by the fact that, even if anomalies are few in comparison to normal examples and, moreover, even if anomalies would globally weight less than inliers in the loss function, the reconstruction error of a badly reconstructed anomaly (that is not reversely reconstructed as required by our loss) is potentially much larger than the reconstruction error associated with a single inlier. Indeed, if the network learned to output the original reconstruction of even a single anomaly, the associated loss contribution would be very large due to the requirement to minimize the distance to the target reverse reconstruction function. Informally, we can say that AE–SAD cannot afford to make a mistake in reconstructing anomalies. From the above analysis, as for the parameter λ in the following, if not otherwise stated, we hold fixed λ to the value $\lambda_1 = \lambda(0.1)$.

Figure 4 shows the sensitivity to the parameter s on Fashion-MNIST. The AUC of the baseline AE is also reported for comparison. The latter AUC is constant since AE in an unsupervised method and do not uses the labeled anomalies. These curves confirm the behavior already observed on MNIST. Summarizing, as for the number of labeled anomalies s, AE–SAD performs remarkably well even when only a few anomalies are available. The same reason provided above can be used to justify this additional result. In the following, if not otherwise stated, we set $s = 8$ by default to considered the challenging scenario in which a limited number of labeled anomalies is available.

4.3 Sensitivity analysis to the number of epochs

The number training epochs is an important hyperparameter for all methods that employ neural networks. In the anomaly detection context the choice of this hyperparameter might become critical since the value of the loss function of a standard AE is not indicative of the quality as anomaly detector. Indeed, while a low MSE value on the training set means that training examples are well reconstructed,

this does not necessarily implies that anomalies are better identified. Differently, with our method we introduce a novel loss and a novel strategy to train an Autoencoder that aims at enlarging the difference between the reconstruction error of the anomalies and the ones of the normal items.

Thus, in order to compare the behaviour of AE–SAD with standard AEs as the number of epochs grows, we consider the ODDS datasets and we train both the models for 2500 epochs and at each epoch we compute the AUC on the test set and the value of the loss function on the training set. Since the ODDS datasets are not partitioned into a training and a test set by design, we split each of them by randomly selecting the 60% of the normal items to be part of the training set and keeping the rest for the test set. As for the anomalies, we randomly pick a number of them such that they represent the 5% of the training set, while the remaining are used in the test set. For AE–SAD, we fix λ by following the rule obtained in previous section.

The results reported in Figure 5 show that in general AE–SAD outperforms the standard AE in terms of AUC at each epoch. Moreover the trend of the AUC of our method is much more regular and almost always increasing. This can be justified because after a certain number of epochs the standard AE learns to reconstruct also the anomalies, while by using the loss (2) we always increase the contrast between the reconstruction of the two classes during all the training process. It can be concluded that, while the number of epochs is a critical hyperparameter for standard AE, this is not the case for AE–SAD. In our case it is not hard to select a good value for this hyperparameter, since the more epochs we consider, the higher the expected performances.

Legends of Figure 5 report between brackets the AUC value in the epoch in which the lowest value of the loss is reached. As we can see, while for AE–SAD a low value of the loss always implies an AUC value close to the maximum, the same fact does not hold for standard AE. Indeed, for what concerns the latter method, often the maximum AUC value is obtained when the value of the loss is still high. This happens because, the standard AE is able to generalize to classes not included in the training set that could share similarities with anomalies, while the same effect is mitigated in AE–SAD by the nature of the loss.

Finally, from Figure 5 we can deduce that in general AE–SAD achieves good AUC values even after a relatively small number of epochs, thus the training is not too slow.

4.4 Sensitivity analysis to the function $F(\mathbf{x})$

In this section we investigate the behavior of AE–SAD with different definitions of the function $F(\mathbf{x})$.

In particular, we consider the following functions:

- $F_0(\mathbf{x}) = \mathbf{1} - \mathbf{x}$ which corresponds, in the domain of the images, to the negative image of $\mathbf{x}$.
- $F_1(\mathbf{x}) = \mathbb{1}_{[0,\frac{1}{2}]}(\mathbf{x})$ that is the function that maximizes the difference between the input and the output.
- $F_2(\mathbf{x}) = (\mathbf{x} + \frac{1}{2}\mathbf{1})\mathbb{1}_{[0,\frac{1}{2}]}(\mathbf{x}) + (\mathbf{x} - \frac{1}{2}\mathbf{1})\mathbb{1}_{(\frac{1}{2},1]}(\mathbf{x})$ that has the property that the absolute difference between a point and its output is constantly equal to $\frac{1}{2}$.

Figure 6 reports the plots of the three functions F_0, F_1 and F_2 (on the left), as well as their absolute (on the center) and squared (on the right) differences with the input. These functions are relevant because of the following reasons:

- F_0 is continuous and its squared difference is differentiable, while it is not the case for F_1 and F_2;
- as for F_1, both the absolute and squared differences between the original and the transformed value are maximized compared with the other two definitions;
- as for F_2, the absolute and squared differences between the original and the transformed value are constantly equal to $\frac{1}{2}$ and $\frac{1}{4}$, respectively, thus all the attribute values of an anomalous example provide the same contribution to the anomaly score.

Table 2 reports the results of the comparison between these three functions on MNIST, Fashion MNIST and CIFAR-10. It can be observed that the function F_0, used as the default F function in Equation (2), scores the highest number of wins in the considered tests, F_1 is the runner up, while F_2 seems to present lower performances. From these results we can draw the conclusion that the property of F_2 to vary each value of the same entity has a lower impact on the accuracy of the technique than the property of maximizing the amount of the difference. As for F_1, it shows intermediate performances since, on one hand, it is able to make larger variations than F_0 and, in particular, it does not cause variations below $\frac{1}{2}$; on the other hand, both F_1 and F_2 present a discontinuity making them harder to be learned than F_1, so performances are degraded.

We further note that functions F_1 and F_2 share the property of mapping any value to a value located at a distance not less than $\frac{1}{2}$ from the original one. Particular settings, such as those in which examples are affected by fog, can be usually satisfactorily managed by using the default function F_0 with a normalization pre-processing step. However, if this does not help, F_1 and F_2 can be suitably used to deal with data distributed around the value $\frac{1}{2}$. Indeed, we simulated a scenario of low contrasting images by uniformly scaling the dataset attributes in the range $[0.4, 0.6]$ and considered the 0 as normal class, obtaining the following results: $AUC(F_0) = 0.9766 \pm 0.0019$, $AUC(F_1) = 0.9841 \pm 0.0054$, and $AUC(F_2) = 0.9877 \pm 0.0022$.

4.5 Comparison with other methods

In this section, we compare AE–SAD with the baseline AE, OC-SVM, A^3, Deep-SAD, Neg-AE, and DRA. First, we test them on the ODDS datasets splitting them into training and test set in the same way as previous section. Since the selection of both anomalies and normal examples for training and test set is done by random sampling, we tested all the considered methods on 5 different runs for each dataset and we compute the mean and the standard deviation of the AUC. The results, reported in the Table 3 show that our method almost always achieves better performances than competitors, and sometimes this improvements are relevant. Note that DRA does not appear in Table 3 since it is specific for images and cannot be applied to tabular data.

We also consider for the comparison with the competitors the MNIST, Fashion-MNIST and CIFAR-10 datasets, for which we implement the one-vs-one setting already illustrated in Section 4.2. This setting simulates the scenario

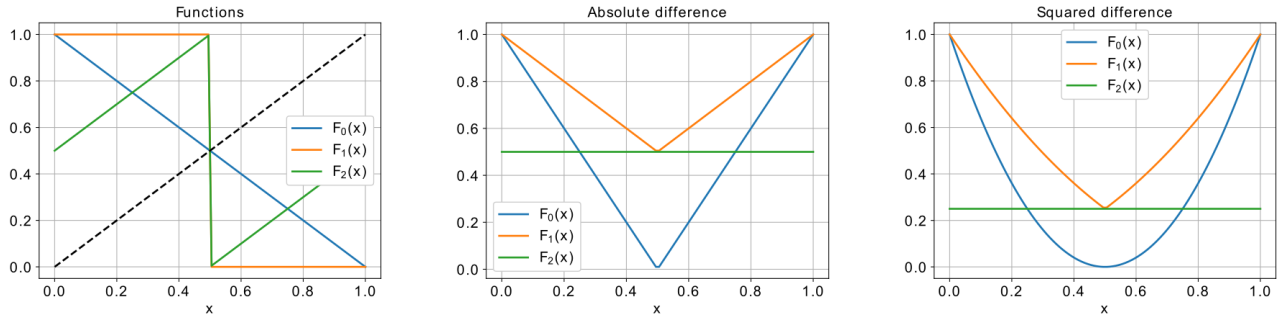


Fig. 6. Plots of the functions $F_i(\mathbf{x})$, $|F_i(\mathbf{x}) - \mathbf{x}|$ and $(F_i(\mathbf{x}) - \mathbf{x})^2$ for $i = 0, 1, 2$.

Dataset	MNIST			Fashion MNIST			CIFAR-10		
Class	$F_0(\mathbf{x})$	$F_1(\mathbf{x})$	$F_2(\mathbf{x})$	$F_0(\mathbf{x})$	$F_1(\mathbf{x})$	$F_2(\mathbf{x})$	$F_0(\mathbf{x})$	$F_1(\mathbf{x})$	$F_2(\mathbf{x})$
0	**.998±.00**	**.998±.00**	.995±.00	**.952±.00**	.951±.01	.943±.01	**.700±.03**	.655±.08	.672±.02
1	**.999±.00**	**.999±.00**	**.999±.00**	**.994±.00**	.993±.00	.991±.00	.490±.05	.463±.06	**.490±.04**
2	.979±.01	**.983±.01**	.971±.01	.921±.01	**.923±.01**	.916±.01	.646±.03	.647±.03	**.668±.01**
3	**.982±.00**	.975±.01	.974±.00	.954±.00	**.955±.00**	.952±.00	**.534±.05**	.526±.04	.516±.02
4	.986±.00	**.987±.00**	.980±.00	**.923±.00**	.921±.00	.922±.00	.725±.02	.706±.01	**.729±.02**
5	**.988±.00**	.987±.00	.980±.00	**.930±.02**	.924±.02	.911±.01	.558±.03	**.589±.02**	.567±.04
6	**.998±.00**	**.998±.00**	.992±.00	**.854±.01**	.853±.01	.845±.00	**.752±.01**	.669±.07	.741±.02
7	.986±.00	**.988±.00**	.983±.00	**.988±.00**	.987±.00	.986±.00	.515±.03	**.539±.04**	.524±.02
8	.955±.01	**.965±.01**	.928±.01	**.969±.00**	.968±.01	.958±.01	**.753±.03**	.687±.06	.741±.04
9	**.982±.00**	.982±.00	.976±.00	**.983±.00**	.983±.00	.981±.00	.482±.05	.468±.09	**.526±.06**

TABLE 2
Comparison between three different versions of AE–SAD employing F_0, F_1 and F_2 on MNIST, Fashion-MNIST and CIFAR-10.

ODDS Datasets

Dataset (d)	AE–SAD	AE	OC-SVM	A^3	Deep-SAD	Neg-AE
annt. (6)	**.990±.00**	.692±.10	.585±.00	.689±.08	.662±.04	.886±.06
arrh. (274)	.796±.05	.789±.01	**.808±.02**	.692±.04	.751±.04	.497±.03
breastw (9)	.992±.00	**.995±.00**	.994±.00	**.995±.00**	.977±.01	.989±.01
cardio (21)	**.994±.00**	.857±.08	.954±.00	.904±.02	.785±.07	.969±.01
glass (9)	**.974±.01**	.664±.11	.527±.18	.639±.10	.880±.08	.908±.04
iono. (33)	**.971±.01**	.910±.01	.918±.01	.668±.01	.953±.01	.537±.07
letter (32)	**.908±.03**	.802±.02	.546±.02	.539±.02	.768±.02	.444±.03
lympho (18)	.986±.01	**.993±.01**	.940±.04	.977±.04	.860±.16	.965±.04
mamm. (6)	**.916±.03**	.874±.02	.880±.02	.835±.05	.759±.11	.879±.07
musk (166)	**1.00±.00**	**1.00±.00**	**1.00±.00**	**1.00±.00**	**1.00±.00**	**1.00±.00**
optd. (64)	**1.00±.00**	.975±.01	.718±.03	.906±.03	.843±.05	.997±.00
pend. (16)	**1.00±.00**	.747±.08	.983±.01	.983±.01	.964±.02	.999±.00
pima (8)	**.733±.02**	.594±.03	.698±.01	.634±.05	.657±.02	.576±.10
satellite (36)	**.907±.02**	.812±.01	.728±.00	.802±.00	.827±.03	.773±.02
sat. (36)	**.999±.00**	.998±.00	.997±.00	.982±.00	.968±.01	.998±.00
shuttle (9)	.808±.35	.914±.14	**.993±.00**	.977±.01	.984±.01	.979±.01
speech (400)	.479±.02	.463±.02	.465±.02	.535±.05	.478±.03	**.622±.03**
thyroid (6)	**.987±.01**	.922±.07	.897±.04	.776±.12	.885±.09	.963±.06
vertebral (6)	.635±.14	.444±.08	.522±.06	.351±.03	.558±.04	**.746±.03**
vowels (12)	**.975±.02**	.904±.03	.816±.05	.528±.06	.937±.05	.832±.11
wbc (30)	.970±.02	.964±.02	.953±.02	.878±.05	.891±.06	**.983±.02**
wine (13)	.994±.00	.829±.12	**.998±.00**	.893±.08	.817±.14	.982±.02

TABLE 3
Comparison with competitors in the ODDS datasets.

in which the detector has limited knowledge of the expected anomalies and, as such, it is a notable scenarios to deal with.

Table 4 reports the relative number of wins for each pair of methods on the MNIST, Fashion MNIST, and CIFAR-10 datasets. Specifically, each entry of the table is the probability that the method located on the corresponding row scores an AUC greater than that of the method located on the corresponding column. In the considered setting AE–SAD shows a larger probability of winning over all the competitors on all the datasets. Neg-AE is the runner-up

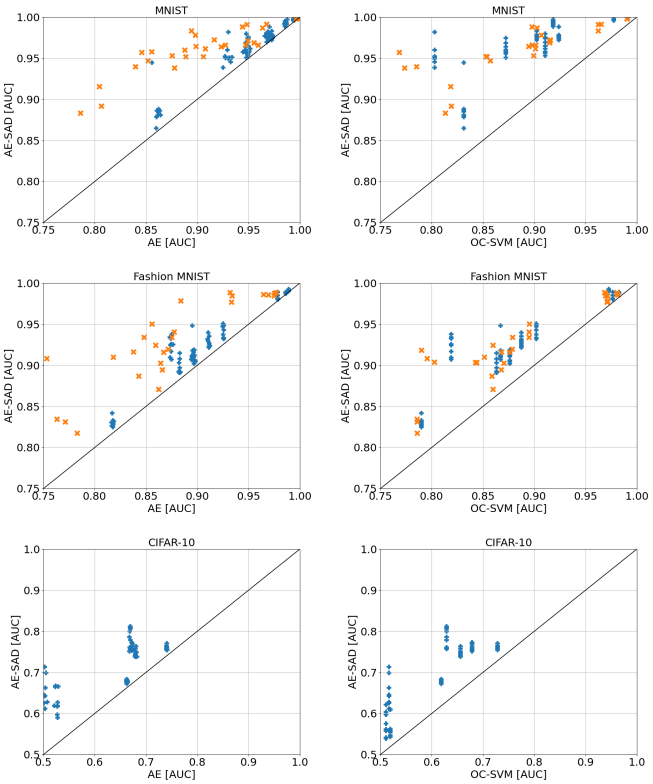


Fig. 7. Comparison of the AUCs of AE–SAD and competitors on both clean (blue +-marks) and polluted (red ×-marks) data. Group 1.

in terms of number of wins, and also Deep-SAD achieve sufficient performances, while A^3 and DRA do not perform well in this setting despite being semi-supervised methods.

MNIST

	AE-SAD	*OC-SVM*	*AE*	A^3	*Deep-SAD*	*Neg-AE*	*DRA*	*mean*
AE–SAD	—	**1.00**	**0.91**	**1.00**	**0.85**	**0.79**	**0.96**	**0.92**
OC-SVM	0.00	—	0.00	**0.86**	0.05	0.01	**0.72**	0.27
AE	0.09	**1.00**	—	**1.00**	**0.70**	**0.51**	**0.93**	0.70
A^3	0.00	0.14	0.00	—	0.02	0.00	0.49	0.11
Deep-SAD	0.15	**0.95**	0.30	**0.98**	—	0.24	**0.91**	0.59
Neg-AE	0.21	**0.99**	0.49	**1.00**	**0.76**	—	**0.79**	0.71
DRA	0.04	0.28	0.07	**0.51**	0.09	0.21	—	0.20
mean	**0.08**	0.73	0.30	0.89	0.41	0.29	0.80	—

Fashion MNIST

	AE-SAD	*OC-SVM*	*AE*	A^3	*Deep-SAD*	*Neg-AE*	*DRA*	*mean*
AE–SAD	—	**1.00**	**0.98**	**0.98**	**0.64**	**0.51**	**0.96**	**0.85**
OC-SVM	0.00	—	0.01	**0.90**	0.11	0.02	**0.91**	0.33
AE	0.02	**0.99**	—	**0.97**	0.38	0.17	**0.94**	0.58
A^3	0.02	0.10	0.03	—	0.03	0.02	**0.72**	0.15
Deep-SAD	0.36	**0.89**	0.62	**0.97**	—	0.38	**0.95**	0.70
Neg-AE	0.49	**0.98**	**0.83**	**0.98**	**0.62**	—	**0.79**	0.78
DRA	0.04	0.09	0.06	0.28	0.05	0.21	—	0.12
mean	**0.15**	0.67	0.42	0.85	0.30	0.22	0.88	—

CIFAR-10

	AE-SAD	*OC-SVM*	*AE*	A^3	*Deep-SAD*	*Neg-AE*	*DRA*	*mean*
AE–SAD	—	**0.97**	**1.00**	**0.86**	**0.81**	**0.73**	**0.80**	**0.82**
OC-SVM	0.03	—	0.48	0.39	0.29	0.26	**0.67**	0.35
AE	0.00	**0.52**	—	**0.54**	0.42	0.26	**0.63**	0.39
A^3	0.14	**0.61**	0.46	—	0.48	0.04	0.06	0.30
Deep-SAD	0.19	**0.71**	**0.58**	**0.52**	—	0.05	0.08	0.35
Neg-AE	0.27	**0.74**	**0.74**	**0.96**	**0.95**	—	**0.67**	0.72
DRA	0.20	0.33	0.37	**0.94**	**0.92**	0.33	—	0.51
mean	**0.18**	0.65	0.61	0.70	0.65	0.28	0.49	—

TABLE 4
Probability of winning for each pair of methods in the one-vs-one experimental setting.

In Section 4.8, we compare AE–SAD and A^3 on a setting where competitor performance are shown in [27] to be strong across various configurations.

Table 5 details the result for each pair of normal and known anomalous classes, by reporting the relative number of wins of AE–SAD on the competitors. Specifically, each entry of the table is the probability that the AUC scored by AE–SAD is greater than the AUC scored by another method. For almost all the combinations AE–SAD is preferable to a randomly selected method of the pool.

To provide details also on the comparison of the methods on the each single dataset, Figures 7, 8, 9 reports the scatter plot of the AUC of AE–SAD (on the y-axis) vs the AUC of each competitor (on the x-axis) associated with each of the above experiments. Specifically, blue $+$-marks are relative to the experiments described in this section, while red $\times$-marks are relative experiments reported in the next section concerning polluted normal data.

MNIST

N \ A	0	1	2	3	4	5	6	7	8	9	μ
0	–	1.00	1.00	1.00	0.97	1.00	1.00	0.95	1.00	0.98	0.99
1	0.95	–	0.90	0.95	1.00	0.98	0.95	1.00	0.97	1.00	0.97
2	0.77	1.00	–	0.83	0.87	0.87	0.72	0.85	0.93	0.88	0.86
3	0.85	0.98	0.97	–	0.90	0.97	0.97	0.97	0.98	0.97	0.95
4	0.70	0.97	1.00	0.95	–	0.78	0.63	0.92	0.93	0.83	0.86
5	0.85	1.00	0.97	0.98	0.90	–	0.95	0.97	1.00	0.98	0.96
6	0.78	1.00	0.90	0.93	0.80	0.93	–	0.85	0.95	0.93	0.90
7	0.88	1.00	0.92	0.88	0.92	0.93	0.88	–	0.97	0.98	0.93
8	0.63	0.92	0.82	0.75	0.75	0.82	0.73	0.72	–	0.83	0.77
9	0.83	0.97	1.00	0.97	0.95	0.97	0.93	1.00	0.92	–	0.95
μ	0.81	0.98	0.94	0.92	0.89	0.92	0.86	0.91	0.96	0.93	0.91

Fashion MNIST

N \ A	0	1	2	3	4	5	6	7	8	9	μ
0	–	0.90	0.97	0.92	0.97	0.93	0.95	0.75	0.90	0.83	0.90
1	0.95	–	0.93	0.90	0.93	0.85	0.98	0.95	0.97	0.88	0.93
2	1.00	0.85	–	0.98	0.82	0.80	0.93	0.80	0.85	0.80	0.87
3	0.90	0.83	0.88	–	0.88	0.82	0.93	0.72	0.83	0.72	0.84
4	0.93	0.85	0.77	0.97	–	0.73	0.83	0.60	0.77	0.73	0.80
5	0.75	0.70	0.75	0.78	0.70	–	0.70	0.92	0.72	0.80	0.76
6	0.98	0.92	0.85	0.98	0.93	0.85	–	0.78	1.00	0.90	0.91
7	0.88	0.82	0.85	0.83	0.90	0.93	0.88	–	0.83	0.88	0.87
8	0.77	0.62	0.77	0.75	0.83	0.63	0.78	0.68	–	0.65	0.72
9	0.68	0.77	0.65	0.77	0.67	0.78	0.78	0.98	0.68	–	0.75
μ	0.87	0.81	0.82	0.88	0.85	0.81	0.86	0.80	0.84	0.80	0.83

CIFAR-10

N \ A	0	1	2	3	4	5	6	7	8	9	μ
0	–	0.83	0.93	0.92	0.98	0.92	0.98	0.95	0.77	0.88	0.91
1	0.82	–	0.80	0.78	0.92	0.85	0.83	0.75	0.78	0.65	0.80
2	0.95	1.00	–	0.98	0.98	0.93	0.95	0.93	1.00	0.97	0.97
3	0.83	0.95	0.88	–	0.90	0.87	0.87	0.87	0.93	0.98	0.90
4	0.98	0.98	0.97	0.95	–	0.95	1.00	0.98	1.00	0.97	0.98
5	0.88	0.87	0.73	0.75	0.88	–	0.62	0.65	0.95	0.85	0.80
6	1.00	0.98	0.98	0.98	1.00	0.98	–	1.00	0.98	0.98	0.99
7	0.88	0.75	0.87	0.93	0.87	0.68	0.78	–	0.97	0.75	0.83
8	0.83	0.85	0.95	0.85	0.98	0.95	0.97	0.93	–	0.87	0.91
9	0.65	0.55	0.88	0.70	0.85	0.75	0.70	0.68	0.68	–	0.72
μ	0.87	0.86	0.89	0.87	0.93	0.88	0.86	0.86	0.90	0.88	0.88

TABLE 5
Probability of winning of AE–SAD for each pair of normal and anomalous classes in the one-vs-one setting.

4.6 Behaviour varying the image resolution

In order to investigate how AE–SAD performances vary with images of different size, we consider the FRUIT dataset [49]. It is composed of high resolution images of 15 types of fruit, each of which represents a distinct class. Each class consists of at least two thousands examples (on the average about 3000 items per class). We consider a *one-vs-all* strategy, in which the training set is built by considering one class as normal and by randomly sampling 50 anomalies from the other classes. Table 6 details the comparison between AE–SAD applied to the images scaled at different sizes (namely 16×24, 32×48, 64×96, and 128×192 have been considered) and the competitors applied to the images with size 128×192. We can observe that high resolution images do not cause performance degradation. On the contrary, as reported in Table 6, AE–SAD benefits of larger resolution inputs, since the AUC is increasing with the image size. Moreover, AE–SAD outperforms all the competitors even when it deals with high resolution images.

4.7 Normal data polluted

In real scenarios it may happen that the training set is polluted by anomalies that are mislabeled and thus appear to the model as normal items. E.g. consider the cases in which the normal data is collected by exploiting semi-automatic or

Methods	AE–SAD				AE	OC-SVM	A^3	Deep-SAD	Neg-AE	DRA
Class	16×24	32×48	64×96	128×192	128×192					
Apple	.648±.03	.810±.03	.916±.02	**.939±.01**	.473±.01	.309±.00	.629±.04	.516±.04	.468±.01	.747±.08
Banana	.877±.01	.899±.01	.956±.01	**.984±.01**	.611±.02	.405±.00	.640±.17	.491±.05	.554±.08	.660±.16
Carambola	.869±.03	.940±.04	**.979±.02**	**.979±.02**	.907±.01	.360±.00	.478±.07	.462±.04	.653±.05	.926±.03
Guava	.912±.04	.979±.01	.984±.01	**.997±.00**	.953±.01	.934±.00	.726±.07	.784±.05	.894±.07	.722±.12
Kiwi	.988±.00	.992±.00	**.996±.00**	.995±.00	.955±.00	.942±.00	.805±.07	.783±.04	.970±.01	.858±.20
Mango	.891±.03	.889±.06	.936±.01	**.949±.02**	.579±.02	.572±.00	.813±.07	.647±.05	.543±.02	.540±.08
Muskmelon	.965±.01	.985±.01	**.994±.00**	**.994±.00**	.893±.00	.776±.00	.851±.02	.733±.04	.563±.09	.948±.02
Orange	.989±.00	.997±.00	**.999±.00**	.998±.00	.771±.01	.493±.00	.707±.10	.703±.04	.641±.02	.783±.15
Peach	.973±.00	.988±.00	.992±.00	**.997±.00**	.963±.00	.901±.00	.757±.04	.802±.02	.846±.06	.837±.12
Pear	.935±.03	.935±.02	.966±.03	**.974±.02**	.700±.01	.631±.00	.721±.07	.679±.03	.643±.05	.756±.08
Persimmon	.978±.01	.985±.01	.997±.01	**1.00±.00**	.563±.01	.585±.00	.905±.04	.596±.05	.494±.01	.961±.02
Pitaya	.970±.02	.991±.00	.994±.01	**.998±.00**	.425±.01	.522±.00	.896±.04	.667±.01	.487±.07	.734±.13
Plum	.998±.00	**1.00±.00**	**1.00±.00**	**1.00±.00**	.988±.00	.826±.00	.901±.02	.820±.04	.792±.13	.872±.24
Pomegranate	.964±.01	.982±.01	.988±.01	**.994±.00**	.359±.01	.612±.00	.926±.02	.699±.05	.428±.01	.822±.10
Tomatoes	.944±.01	.976±.01	.990±.00	**.995±.00**	.327±.01	.251±.00	.805±.10	.580±.05	.437±.08	.830±.05

TABLE 6
AE–SAD applied to the FRUIT dataset scaled at different dimensions and compared with comparison on the size 128×192.

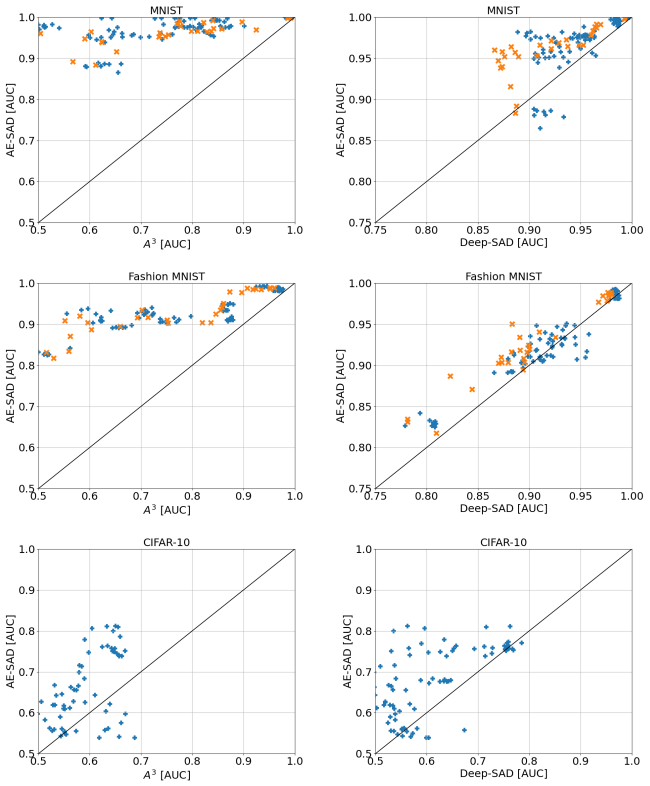


Fig. 8. Comparison of the AUCs of AE–SAD and competitors on both clean (blue $+$-marks) and polluted (red $\times$-marks) data. Group 2.

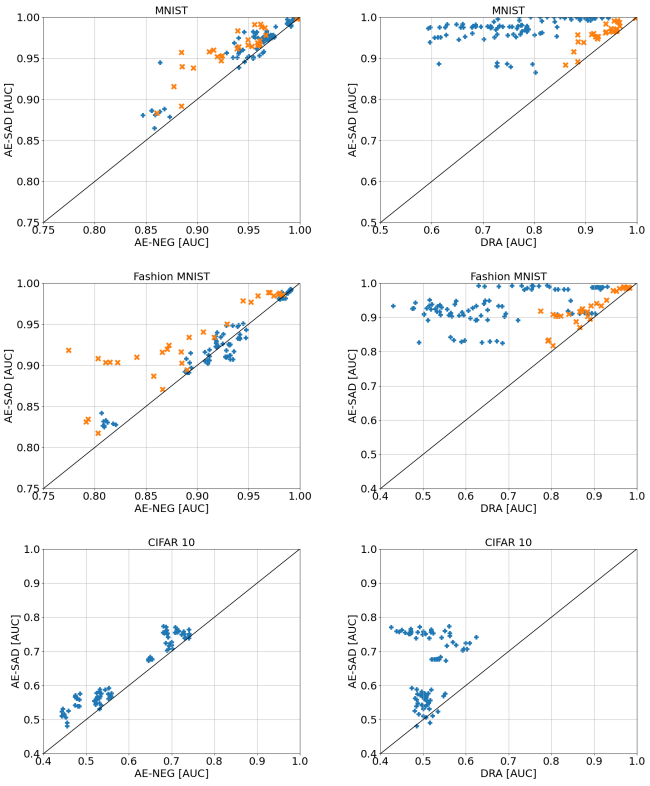


Fig. 9. Comparison of the AUCs of AE–SAD and competitors on both clean (blue $+$-marks) and polluted (red $\times$-marks) data. Group 3.

not completely reliable procedures and/or limited resources are available for cleaning the possibly huge normal data collection, and an human analyst points out to the system a small number of selected anomalies in order to improve its performances. In this section we investigate the above scenario. We pollute the datasets by injecting a certain percentage of mislabeled known anomalous examples (these examples are wrongly labeled as 0). The *percentage level of pollution* quantifies the number of mislabeled known anomalies as a percentage of the number of true inliers in the dataset. That is to say, a 5% level of pollution means that in the training set there are $0.05n$ mislabeled known anomalies (i.e., labeled 0) and n inliers (also labeled 0).

Because of the shape of the loss (2) the Autoencoder is trained to well reconstruct the mislabeled anomalies, it is important to select a value of the hyper-parameter λ that forces the model to give more relevance to the (bad) reconstruction of the anomalies labeled 1. In Figure 10 we show the results obtained for different values of λ. The number s of correctly labeled known anomalies is set to $s = 100$. The red lines refer to the 2% percentage level of pollution, while the blue lines to the 4% of pollution. We can observe that increasing values of λ tend to mitigate the effect of the pollution and to obtain better results. Despite this, the choice of a good value for λ is not critical, since all the values of λ considered guarantee good performances,

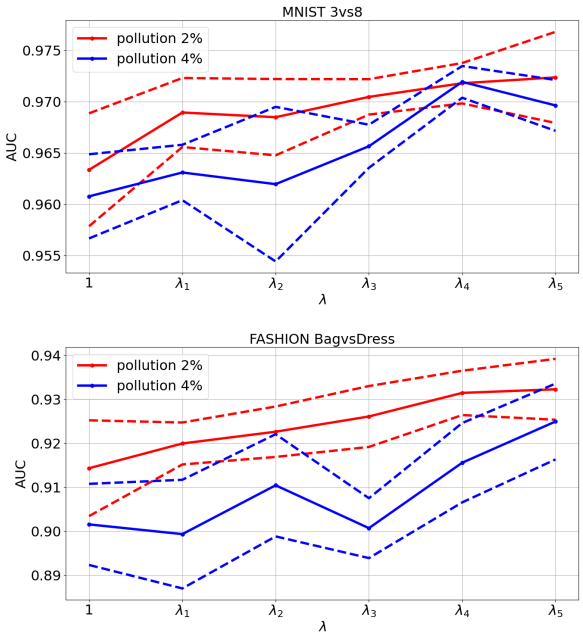


Fig. 10. Sensitivity analysis of λ for different levels of pollution.

MNIST 3vs[4,5,6]

Method	Pollution				
	0.25%	0.5%	2.5%	5%	12.5%
AE–SAD	**.968±.01**	**.966±.00**	**.957±.01**	**.961±.01**	**.955±.00**
Standard AE	.944±.01	.942±.00	.919±.01	.904±.01	.864±.01
OC-SVM	.871±.00	.870±.00	.863±.00	.854±.00	.821±.00
A^3	.622±.04	.675±.03	.725±.10	.653±.08	.603±.08
Deep-SAD	.958±.10	.957±.00	.948±.00	.944±.01	.911±.03
AE-Neg	.940±.00	.942±.00	.933±.00	.926±.00	.910±.10
DRA	.941±.05	.908±.07	.909±.01	.893±.05	.785±.16

Fashion MNIST 3vs[7,8,9]

Method	Pollution				
	0.25%	0.5%	2.5%	5%	12.5%
AE–SAD	**.946±.00**	**.944±.00**	**.945±.00**	**.942±.00**	**.939±.00**
Standard AE	.917±.00	.908±.00	.890±.00	.880±.00	.858±.00
OC-SVM	.902±.00	.901±.00	.898±.00	.895±.00	.880±.00
A^3	.877±.01	.875±.01	.861±.01	.852±.01	.838±.01
Deep-SAD	.925±.01	.913±.01	.904±.02	.905±.01	.901±.01
AE-Neg	.941±.00	.939±.00	.932±.01	.921±.00	.887±.02
DRA	.774±.09	.681±.05	.631±.11	.576±.11	.456±.14

TABLE 7
Comparison with competitors for various percentage levels of pollution.

though the largest ones are generally associated with the best scores. In the sequel of this subsection we set λ to about n/s which corresponds approximately to λ_4.

In Table 7 we compare our method with other anomaly detection algorithms for various percentages of pollution. The one-vs-three setting is considered here to increase the diversity of the pollution. The number of correctly labeled known anomalies is $s = 150$ (50 for each of the three known anomalous classes). AE–SAD improves on the baseline AE and performs better also in this setting.

In order to make the results more robust, we next build up two systematic experiments. In each one, we add to the inliers in the training set 100 randomly selected elements for each anomalous class. Moreover, we assume that only 2 examples per anomalous class are correctly labeled 1.

The first experiment concerns the *one-vs-all scenario* in

MNIST

Class	AE–SAD	AE	OC-SVM	A^3	Deep-SAD	Neg-AE	DRA
0	**.980±.01**	.885±.01	*.911±.00*	.770±.02	.910±.04	.903±.01	.609±.27
1	**.996±.00**	*.996±.00*	.986±.00	.983±.04	.983±.01	.996±.00	.367±.28
2	**.913±.02**	.767±.02	.729±.00	.676±.02	*.869±.03*	.827±.01	.464±.16
3	**.929±.01**	.821±.01	.820±.00	.730±.01	.861±.04	*.885±.02*	.540±.15
4	**.940±.01**	.854±.01	.876±.00	.794±.07	.880±.04	*.910±.01*	.557±.14
5	**.915±.01**	.807±.02	.697±.00	.484±.11	.827±.06	*.876±.01*	.616±.12
6	**.958±.01**	.877±.02	.863±.00	.882±.08	.925±.03	*.927±.01*	.632±.13
7	**.956±.02**	.907±.00	.895±.00	.879±.06	.900±.03	*.935±.01*	.606±.20
8	*.843±.02*	.750±.02	.788±.00	.663±.07	**.866±.03**	.824±.02	.598±.18
9	**.942±.02**	.897±.01	.884±.00	.829±.07	.922±.01	*.941±.01*	.573±.19

Fashion MNIST

Class	AE–SAD	AE	OC-SVM	A^3	Deep-SAD	Neg-AE	DRA
0	**.911±.07**	.781±.01	*.859±.00*	.692±.05	.831±.06	.838±.01	.490±.11
1	**.980±.01**	.958±.00	.957±.00	.949±.01	.957±.03	*.962±.00*	.425±.17
2	**.877±.04**	.729±.02	*.843±.00*	.576±.10	.801±.07	.813±.01	.397±.15
3	**.920±.03**	.807±.01	.879±.00	.857±.02	*.904±.02*	.893±.01	.425±.13
4	*.847±.04*	.770±.04	**.853±.00**	.762±.06	.843±.05	.845±.01	.524±.13
5	**.908±.02**	.633±.02	.807±.00	.841±.04	*.864±.01*	.755±.03	.375±.19
6	**.796±.04**	.701±.04	*.778±.00*	.452±.08	.760±.03	.761±.01	.456±.11
7	**.982±.00**	.927±.01	*.975±.00*	.970±.00	.931±.06	.957±.01	.408±.14
8	**.940±.02**	.541±.01	.755±.00	.607±.05	*.878±.05*	.665±.02	.442±.08
9	**.971±.01**	.807±.02	.952±.00	*.966±.02*	.937±.07	.890±.02	.415±.17

TABLE 8
Comparison with competitors in the *one-vs-all* polluted setting.

which we consider all the examples of a selected class as inliers and all the other classes as known anomalous ones. These datasets are extremely polluted, being their pollution percentage about 15%, and anomalies have large diversity. The results, reported in Table 8, show that for each class our method achieves good performances being almost always the best method. Even in those few cases in which it is not the best method, it is always the second ranked with an AUC value very close to the one of the first.

The second experiment considers the *one-vs-many* scenario in which we have three known anomalous classes. Specifically, to make experiments easily reproducible we fix in turn the normal class and partition the remaining nine classes into three groups consisting of three consecutive classes. The pollution percentage in this case is about the 5%. Table 9 reports the result of this experiment, which confirm the generalization ability of AE–SAD in the polluted scenario. In Figures 7, 8, 9, red $\times$-marks compare the AUCs on polluted data of experiments in Tables 8 and 9.

4.8 Behavior in the A^3 experimental setting

In this section, in order to compare AE–SAD and A^3 on a setting where competitor performance are shown in [27] to be strong across various configurations, we consider precisely the one proposed by the A^3 authors where they show how their method well surpasses the unsupervised baseline methods, and on most experiments also the semi-supervised baseline methods. In these experiments both normal examples and anomalies belong to a set of classes, thus can be characterized as a *many-vs-many* scenario, and consider the there employed image data, namely MNIST and E-MNIST. The composition of the datasets, as specified in [27], are reported in Table 10 and the results have been obtained by using the same experimental setting. In this experiments, authors [27] compare their method, namely A^3, with standard AE, Isolation Forest [9], DAGMM [28]

Dataset		MNIST							Fashion MNIST						
N	A	AE–SAD	AE	OC-SVM	A^3	Deep-SAD	Neg-AE	DRA	AE–SAD	AE	OC-SVM	A^3	Deep-SAD	Neg-AE	DRA
0	[1,2,3]	**.983±.01**	.896±.00	.962±.00	.770±.08	.962±.01	.939±.01	.531±.28	**.934±.00**	.849±.00	.879±.00	.702±.04	.891±.02	.892±.01	.445±.14
	[4,5,6]	**.991±.00**	.968±.00	.965±.00	.840±.08	.965±.01	.962±.01	.478±.19	**.920±.00**	.870±.00	.878±.00	.581±.05	.900±.00	.871±.02	.409±.18
	[7,8,9]	**.991±.00**	.948±.00	.962±.00	.775±.06	.969±.00	.956±.01	.418±.23	**.916±.00**	.867±.00	.876±.00	.693±.01	.898±.01	.866±.01	.482±.15
1	[0,2,3]	**.998±.00**	.998±.00	.991±.00	.986±.01	.993±.00	.998±.00	.373±.11	**.988±.00**	.975±.00	.970±.00	.907±.04	.978±.00	.980±.00	.462±.31
	[4,5,6]	**.998±.00**	.997±.00	.990±.00	.992±.00	.993±.00	.998±.00	.651±.31	**.984±.00**	.974±.00	.969±.00	.934±.01	.977±.00	.974±.00	.341±.13
	[7,8,9]	**.998±.00**	.996±.00	.990±.00	.985±.02	.993±.00	.997±.00	.577±.25	**.988±.00**	.978±.00	.968±.00	.951±.01	.980±.00	.971±.00	.463±.18
2	[0,1,3]	**.940±.01**	.841±.00	.785±.00	.623±.09	.873±.01	.885±.02	.667±.09	**.910±.00**	.861±.00	.851±.00	.751±.02	.873±.01	.841±.01	.375±.20
	[4,5,6]	**.938±.00**	.879±.01	.774±.00	.624±.09	.872±.03	.896±.01	.455±.13	**.871±.01**	.819±.00	.860±.00	.562±.04	.844±.02	.866±.01	.422±.09
	[7,8,9]	**.957±.01**	.844±.01	.768±.00	.753±.10	.886±.02	.885±.01	.665±.10	**.887±.00**	.844±.00	.859±.00	.603±.02	.823±.04	.858±.01	.606±.13
3	[0,1,2]	**.947±.00**	.851±.00	.857±.00	.591±.07	.869±.05	.923±.01	.581±.09	**.950±.00**	.855±.00	.895±.00	.861±.01	.883±.07	.929±.01	.460±.20
	[4,5,6]	**.952±.00**	.905±.01	.854±.00	.736±.05	.889±.02	.919±.01	.443±.18	**.934±.00**	.876±.00	.895±.00	.855±.01	.909±.01	.916±.00	.446±.16
	[7,8,9]	**.952±.01**	.891±.01	.853±.00	.735±.04	.876±.04	.923±.01	.539±.10	**.941±.01**	.879±.00	.895±.00	.858±.01	.926±.02	.906±.01	.366±.11
4	[0,1,2]	**.953±.01**	.875±.00	.899±.00	.747±.05	.907±.01	.924±.01	.593±.11	**.916±.00**	.837±.00	.868±.00	.714±.05	.882±.01	.884±.01	.380±.16
	[3,5,6]	**.966±.00**	.926±.00	.901±.00	.798±.07	.910±.02	.949±.00	.679±.10	**.895±.01**	.865±.00	.868±.00	.660±.05	.870±.01	.890±.00	.530±.13
	[7,8,9]	**.961±.01**	.906±.00	.901±.00	.737±.06	.921±.03	.939±.01	.700±.16	**.903±.00**	.864±.00	.870±.00	.753±.02	.894±.03	.885±.01	.299±.09
5	[0,1,2]	**.958±.01**	.856±.00	.731±.00	.482±.09	.873±.01	.912±.02	.552±.20	**.903±.00**	.715±.01	.844±.00	.836±.01	.873±.01	.811±.01	.394±.11
	[3,4,6]	**.964±.00**	.916±.01	.725±.00	.603±.04	.882±.02	.940±.01	.589±.13	**.903±.00**	.746±.00	.843±.00	.819±.03	.879±.01	.822±.03	.413±.26
	[7,8,9]	**.960±.00**	.889±.00	.722±.00	.504±.05	.866±.01	.916±.02	.622±.16	**.925±.01**	.858±.00	.860±.00	.846±.05	.899±.01	.872±.00	.403±.18
6	[0,1,2]	**.978±.00**	.899±.00	.907±.00	.777±.10	.960±.01	.967±.01	.465±.24	**.831±.01**	.771±.00	.786±.00	.516±.03	.781±.04	.792±.01	.596±.21
	[3,4,5]	**.987±.00**	.962±.00	.903±.00	.822±.09	.965±.01	.966±.01	.601±.21	**.818±.01**	.783±.00	.786±.00	.530±.04	.781±.01	.803±.01	.408±.10
	[7,8,9]	**.988±.00**	.943±.00	.898±.00	.897±.05	.963±.00	.963±.00	.653±.07	**.834±.00**	.763±.00	.786±.00	.560±.03	.809±.01	.793±.01	.502±.14
7	[0,1,2]	**.972±.00**	.916±.00	.916±.00	.841±.05	.936±.01	.949±.00	.454±.13	**.989±.00**	.934±.01	.980±.00	.960±.01	.977±.00	.969±.00	.432±.22
	[3,4,5]	**.969±.00**	.956±.00	.915±.00	.924±.02	.929±.01	.959±.00	.551±.07	**.986±.00**	.966±.00	.981±.00	.924±.03	.978±.00	.979±.00	.643±.23
	[6,8,9]	**.971±.00**	.949±.00	.915±.00	.857±.05	.921±.02	.961±.00	.671±.10	**.986±.00**	.970±.00	.981±.00	.951±.02	.980±.00	.983±.00	.340±.22
8	[0,1,2]	**.915±.01**	.804±.01	.819±.00	.652±.04	.882±.02	.877±.00	.453±.10	**.918±.00**	.701±.01	.790±.00	.490±.05	.891±.03	.774±.03	.421±.15
	[3,4,5]	**.892±.01**	.802±.01	.819±.00	.567±.08	.887±.02	.885±.01	.340±.15	**.908±.02**	.753±.01	.795±.00	.552±.06	.895±.02	.803±.02	.441±.09
	[6,7,9]	.883±.01	.782±.01	.813±.00	.612±.94	**.886±.01**	.861±.01	.608±.07	**.904±.02**	.737±.00	.803±.00	.596±.02	.895±.02	.815±.01	.498±.11
9	[0,1,2]	**.964±.00**	.899±.00	.895±.00	.829±.03	.937±.01	.953±.00	.536±.13	**.978±.00**	.886±.01	.971±.00	.873±.05	.972±.01	.945±.01	.604±.08
	[3,4,5]	**.966±.00**	.959±.00	.898±.00	.837±.03	.953±.01	.961±.00	.423±.17	**.977±.00**	.934±.00	.971±.00	.896±.03	.967±.01	.952±.00	.563±.18
	[6,7,8]	**.966±.00**	.948±.00	.898±.00	.810±.07	.948±.01	.959±.00	.597±.10	**.984±.00**	.934±.00	.971±.00	.918±.03	.976±.01	.959±.00	.413±.21

TABLE 9
Comparison with competitors in the *one-vs-many* polluted setting.

Data	Normal	Train Anomaly	Test Anomaly	AE–SAD AUC	A^3 AUC
MNIST	$0,\ldots,5$	6, 7	6, 7	.98±.00	**.99±.00**
	$4,\ldots,9$	0, 1	0,1	**1.0±.00**	**1.0±.00**
	$0,\ldots,5$	6, 7	6, 7, 8, 9	**.92±.01**	.88±.03
	$4,\ldots,9$	0, 1	0, 1, 2, 3	**.96±.01**	.92±.02
E-MNIST	$0,\ldots,5$	$A,\ldots,E$	$A,\ldots,E$	**.99±.00**	**.99±.01**
	$0,\ldots,5$	$A,\ldots,E$	$A,\ldots,E$, $V,\ldots,Z$	**.98±.00**	.96±.01
	$0,\ldots,5$	$V,\ldots,Z$	$V,\ldots,Z$	**.99±.00**	**.99±.00**
	$0,\ldots,5$	$V,\ldots,Z$	$A,\ldots,E$, $V,\ldots,Z$	**.98±.00**	.95±.02

TABLE 10
Comparison with A^3 in the *many-vs-many* setting [27].

and DevNet [26]. Since A^3 overcomes other competitors, we report only A^3 results and refer the interested readers to Table 3 of [27] for the performances of other methods.

It can be noted that performances of A^3 and AE–SAD are comparable when the set of anomalies in the test set belong to the same classes of the known anomalies, while AE–SAD outperforms A^3 in all datasets and experiments when the test set contains anomalies in classes not represented in the training set. This confirms the ability of AE–SAD in detecting anomalies belonging to completely unknown classes, an important requirement of anomaly detection systems.

5 Conclusion

In this work, we deal with the semi-supervised anomaly detection problem in which a limited number of anomalous examples is available and introduce the AE–SAD algorithm, based on a new loss function that exploits the information of the labelled anomalies and allows Autoencoders to learn to reconstruct them poorly. This strategy increases the contrast between the reconstruction error of normal examples and that associated with both known and unknown anomalies, thus enhancing anomaly detection performances. We focus on classical Autoencoders and show that our proposed strategy is able to reach state-of-the-art performances even when built on very simple architectures. We also prove that AE–SAD is very effective in some relevant anomaly detection scenarios, namely the one in which anomalies in the test set are different from the ones in the training set and the one in which the training set is polluted by some mislabeled examples. As a future work we intend to inject the here presented strategy in other reconstruction-based architectures that permit to organize their latent space to guarantee *continuity* and *completeness*, such as VAEs and GANs, to possibly obtain even improved detection performances.

Acknowledgments

We acknowledge the support of the PNRR project FAIR - Future AI Research (PE00000013), Spoke 9 - Green-aware AI, under the NRRP MUR program funded by the NextGenerationEU.

References

[1] Tax, D. & Duin, R. Support Vector Data Description. *Machine Learning* (2004).
[2] Schölkopf, B., Platt, J., Shawe-Taylor, J., Smola, A. & Williamson, R. Estimating the Support of a High-Dimensional Distribution. *Neural Computation*. (2001).

[3] Aggarwal, C. Outlier Analysis. (Springer,2013)
[4] Goodfellow, I., Bengio, Y. & Courville, A. Deep Learning. (MIT Press,2016)
[5] Chalapathy, R. & Chawla, S. Deep Learning for Anomaly Detection: A Survey. (2019)
[6] Hawkins, S., He, H., Williams, G. & Baxter, R. Outlier Detection Using Replicator Neural Networks. *Int. Conf. (DAWAK)*. (2002).
[7] An, J. & Cho, S. Variational autoencoder based anomaly detection using reconstruction probability. (SNU Data Mining Center,2015)
[8] Angiulli, F. CFOF: A Concentration Free Measure for Anomaly Detection. *ACM (TKDD)*. (2020).
[9] Liu, F., Ting, K. & Zhou, Z. Isolation-Based Anomaly Detection. *ACM (TKDD)*. (2012).
[10] Chandola, V., Banerjee, A. & Kumar, V. Anomaly detection: A survey. *ACM Comput. Surv.*. (2009).
[11] Angiulli, F. & Fassetti, F. DOLPHIN: an Efficient Algorithm for Mining Distance-Based Outliers in Very Large Datasets. *ACM (TKDD)*. (2009).
[12] Angiulli, F. & Pizzuti, C. Fast Outlier Detection in Large High-Dimensional Data Sets. *Int. Conf. (PKDD)*. (2002).
[13] Breunig, M., Kriegel, H., Ng, R. & Sander, J. LOF: Identifying Density-based Local Outliers. *Int. Conf. (SIGMOD)*. (2000).
[14] Knorr, E., Ng, R. & Tucakov, V. Distance-Based Outlier: algorithms and applications. *VLDB Journal*. (2000).
[15] Angiulli, F., Fassetti, F. & Ferragina, L. Improving Deep Unsupervised Anomaly Detection by Exploiting VAE Latent Space Distribution. *23rd Int. Conf. DS 2020, Thessaloniki, Greece, 2020*.
[16] Angiulli, F., Fassetti, F. & Ferragina, L. LatentOut: an unsupervised deep anomaly detection approach exploiting latent space distribution. *Machine Learning*. (2022).
[17] Angiulli, F., Fassetti, F. & Ferragina, L. Detecting Anomalies with LatentOut: Novel Scores, Architectures, and Settings. *26th International Symposium ISMIS*. (2022)
[18] Ruff, L., Görnitz, N., Deecke, L., Siddiqui, S., Vandermeulen, R., Binder, A., Müller, E. & Kloft, M. Deep One-Class Classification. *35th ICML 2018*.
[19] Ruff, L., Vandermeulen, R., Görnitz, N., Binder, A., Müller, E., Müller, K. & Kloft, M. Deep Semi-Supervised Anomaly Detection. *8th ICLR 2020*.
[20] Angiulli, F., Fassetti, F., Ferragina, L. & Spada, R. Cooperative Deep Unsupervised Anomaly Detection. *25th Int. Conf. DS 2022*.
[21] Ruff, L., Kauffmann, J., Vandermeulen, R., Montavon, G., Samek, W., Kloft, M., Dietterich, T. & Müller, K. A Unifying Review of Deep and Shallow Anomaly Detection. *Proc. IEEE*. (2021).
[22] Schlegl, T., Seeböck, P., Waldstein, S., Schmidt-Erfurth, U. & Langs, G. Unsupervised Anomaly Detection with Generative Adversarial Networks to Guide Marker Discovery. *25th IPMI, Boone, USA, 2017*.
[23] Liu, Y., Li, Z., Zhou, C., Jiang, Y., Sun, J., Wang, M. & He, X. Generative Adversarial Active Learning for Unsupervised Outlier Detection. *IEEE Trans. Knowl. Data Eng.*. (2020).
[24] Akcay, S., Atapour-Abarghouei, A. & Breckon, T. GANomaly: Semi-Supervised Anomaly Detection via Adversarial Training. (2018)
[25] Schlegl, T., Seeböck, P., Waldstein, S., Langs, G. & Schmidt-Erfurth, U. f-AnoGAN: Fast Unsupervised Anomaly Detection with Generative Adversarial Networks. *Medical Image Analysis*. (2019).
[26] Pang, G., Shen, C., Cao, L. & Hengel, A. Deep Learning for Anomaly Detection: A Review. *ACM Comput. Surv.* (2021).
[27] Sperl, P., Schulze, J. & Böttinger, K. A^3: Activation Anomaly Analysis. *CoRR*. **abs/2003.01801** (2020).
[28] Zong, B., Song, Q., Min, M., Cheng, W., Lumezanu, C., Cho, D. & Chen, H. Deep Autoencoding Gaussian Mixture Model for Unsupervised Anomaly Detection. *ICLR* (2018).
[29] LeCun, Y. & Cortes, C. MNIST handwritten digit database. (http://yann.lecun.com/exdb/mnist/,2010)
[30] Cohen, G., Afshar, S., Tapson, J. & Schaik, A. EMNIST: an extension of MNIST to handwritten letters. (2017)
[31] Xiao, H., Rasul, K. & Vollgraf, R. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms (2017).
[32] Krizhevsky, A., Nair, V. & Hinton, G. CIFAR-10. http://www.cs.toronto.edu/ kriz/cifar.html
[33] Davies, L. & Gather, U. The identification of multiple outliers. *Journal Of The American Statistical Association*. (1993).
[34] Barnett, V. & Lewis, T. Outliers in Statistical Data. (John Wiley & Sons,1994)
[35] Jin, W., Tung, A. & Han, J. Mining Top-n Local Outliers in Large Databases. *ACM (KDD)*. (2001).
[36] Hautamäki, V., Kärkkäinen, I. & Fränti, P. Outlier Detection Using k-Nearest Neighbour Graph. *(ICPR), Cambridge, UK*. (2004).
[37] Radovanović, M., Nanopoulos, A. & Ivanović, M. Reverse Nearest Neighbors in Unsupervised Distance-Based Outlier Detection. *IEEE (TKDE)*. (2015).
[38] Angiulli, F. Concentration Free Outlier Detection. *(ECMLPKDD), Skopje, Macedonia*. (2017).
[39] Kriegel, H., Schubert, M. & Zimek, A. Angle-based outlier detection in high-dimensional data. *Proc. Int. Conf. On KDD*. (2008)
[40] Rayana, S. ODDS Library. *http://odds.cs.stonybrook.edu* (2017)
[41] Pang, G., Shen, C., Hengel, A. Deep Anomaly Detection with Deviation Networks. *25th ACM SIGKDD Anchorage, USA, 2019*.
[42] Beggel, L., Pfeiffer, M. & Bischl, B. Robust anomaly detection in images using adversarial autoencoders. *ECML PKDD 2019*.
[43] Georgescu, M., Ionescu, R., Khan, F., Popescu, M. & Shah, M. A background-agnostic framework with adversarial training for abnormal event detection in video. *IEEE TPAMI*. (2021)
[44] Astrid, M., Zaheer, M., Lee, J. & Lee, S. Learning not to reconstruct anomalies. *ArXiv Preprint ArXiv:2110.09742*. (2021)
[45] Munawar, A., Vinayavekhin, P. & De Magistris, G. Limiting the reconstruction capability of generative neural network using negative learning. *IEEE 27th International Workshop (MLSP)* (2017)
[46] Zaheer, M., Mahmood, A., Khan, M., Segu, M., Yu, F. & Lee, S. Generative cooperative learning for unsupervised video anomaly detection. *IEEE/CVF Conference On Computer Vision And Pattern Recognition*. (2022)
[47] Acsintoae, A., Florescu, A., Georgescu, M., Mare, T., Sumedrea, P., Ionescu, R., Khan, F. & Shah, M. Ubnormal: New benchmark for supervised open-set video anomaly detection. *IEEE/CVF Conference On Computer Vision And Pattern Recognition*. (2022)
[48] Ding, C., Pang, G. & Shen, C. Catching both gray and black swans: Open-set supervised anomaly detection. *IEEE/CVF Conference On Computer Vision And Pattern Recognition*. (2022)
[49] Xue, G., Liu, S. & Ma, Y. A hybrid deep learning-based fruit classification using attention model and convolution autoencoder. *Complex and Intelligent Systems*. (2020)
[50] Görnitz, N., Kloft, M., Rieck, K. & Brefeld, U. Toward supervised anomaly detection. *Journal Of Artificial Intelligence Research*. (2013)
[51] Liu, W., Luo, W., Li, Z., Zhao, P., Gao, S. & Others Margin Learning Embedded Prediction for Video Anomaly Detection with A Few Anomalies. *IJCAI*. (2019)



Fabrizio Angiulli is a full professor of computer science at DIMES, University of Calabria, Italy. His research interests include data mining, machine learning, and artificial intelligence, with a focus on anomaly detection, large and high-dimensional data analysis, and explainable learning. He has authored more than one hundred papers appearing in premier journals and conferences. He regularly serves on the program committee of several conferences and, as an associate editor of AI Communications.



Fabio Fassetti received the PhD degree in system engineering and computer science in 2008 from the University of Calabria, Italy. He currently is an associate professor of computer engineering at DIMES Department, University of Calabria, Italy. His research interests include bioinformatics, machine learning, data mining, artificial intelligence, knowledge representation and reasoning.



Luca Ferragina received the PhD in Information and Communication Technology from the University of Calabria, Italy. He currently is assistant professor at DIMES Department, University of Calabria, Italy. The main topics of his research are about Anomaly Detection and Deep Learning.